



Universidad de Guadalajara

Centro Universitario de Ciencias Exactas e Ingenierías.

DIVISIÓN DE TECNOLOGÍAS PARA LA INTEGRACIÓN CIBER - HUMANA

Proyecto Final

Nombre del profesor:

Guevara Galindo Gilberto Simon

Equipo 6:

Medina Torres Erick Zaid

Saucedo Aguilar Carlos Andrei

Canales Durán Brandon

Landaverde Jacobo Erick

Davalos Villegas Jesus Daniel

Materia:

Seminario Integración: Protocolo

Sección:

D03

1. Índice

1. Introducción.....	2
2. Planteamiento del problema.....	3
3. Objetivos.....	5
4. Justificación.....	6
5. Estado del arte.....	7
6. Hipótesis.....	10
7. Diseño.....	11
8. Implementación.....	23
9. Conclusiones.....	27
10. Referencias bibliográficas.....	28

Introducción

El control de inventario es un factor determinante para la rentabilidad y la supervivencia en el panorama comercial moderno. Sin embargo, para las Micro y Pequeñas Empresas (MiPyMEs) y los negocios en crecimiento, el acceso a soluciones tecnológicas adecuadas presenta un obstáculo significativo. La mayoría de los *software* de Gestión de Inventarios (SGI) disponibles en el mercado exigen inversiones elevadas, infraestructura compleja (servidores y bases de datos) y un conocimiento técnico especializado, barreras insuperables para el empresario con recursos limitados.

Esta limitación ha perpetuado la dependencia de métodos manuales—hojas de cálculo sin control, registros físicos o informales—, lo que conduce inevitablemente a errores humanos, inconsistencias, desabasto inesperado y sobreinventario. Estas inefficiencias no solo resultan en pérdidas económicas cuantificables, sino que también limitan drásticamente la capacidad de la empresa para tomar decisiones estratégicas informadas y responder ágilmente a la demanda del cliente.

Ante esta realidad, el presente proyecto propone el desarrollo de un Sistema de Gestión de Inventarios (SGI) programado en C++. El enfoque principal es la accesibilidad, la eficiencia y el bajo costo de implementación. El sistema busca ser una herramienta ligera y robusta que simplifique las operaciones básicas de inventario (registro, consulta, modificación y eliminación de datos).

Los alcances de esta solución radican en una decisión técnica clave: utilizar archivos de texto plano (.txt) para el almacenamiento de la información. Esta arquitectura elimina la necesidad de costosos motores de bases de datos o infraestructura compleja, facilitando su uso en equipos de cómputo convencionales y reduciendo el costo total de propiedad (TCO).

En resumen, este SGI en C++ busca democratizar la gestión digital al ofrecer una alternativa funcional que garantice la precisión y la transparencia de los datos de inventario. El proyecto se justifica como un apoyo estratégico que permitirá a las MiPyMEs superar las inefficiencias operativas y asegurar la base informativa necesaria para su crecimiento sostenido y competitivo.

Planteamiento del problema

En la actualidad, muchas empresas pequeñas y en proceso de crecimiento enfrentan dificultades significativas para llevar un control adecuado de sus inventarios. Aunque existen soluciones de software avanzadas en el mercado, la mayoría de estas requieren inversiones elevadas, infraestructura compleja o conocimientos técnicos especializados que no todas las micro y pequeñas empresas poseen. Como resultado, la administración del inventario suele realizarse de forma manual, ya sea mediante hojas de cálculo, documentos escritos o registros informales, lo que provoca inconsistencias, duplicidad de datos y un rezago operativo que afecta directamente la productividad. Esta problemática se intensifica en negocios que dependen del manejo constante de insumos o productos, donde la exactitud en el control del inventario determina la capacidad de respuesta ante sus clientes y la eficiencia general de sus operaciones.

El problema principal radica en que, al no contar con un sistema estructurado que automatice o facilite la gestión del inventario, las empresas se ven expuestas a errores humanos durante el registro de entradas y salidas de productos. Estos errores se traducen en pérdidas económicas, desabasto inesperado, sobreinventario, retrasos en los procesos de compra y una desorganización generalizada que limita la capacidad de la empresa para tomar decisiones informadas. Además, la ausencia de mecanismos de almacenamiento digital confiables genera dificultades al momento de consultar el historial de movimientos o identificar tendencias, lo que impide llevar un control claro del comportamiento del inventario a lo largo del tiempo.

Un problema derivado de esta situación es la falta de uniformidad en los registros, ya que cada empleado puede manejar la información de manera distinta, ocasionando discrepancias y confusiones al momento de verificar existencias reales. Asimismo, la carencia de un respaldo digital estructurado genera vulnerabilidad en la información, ya que el extravío, deterioro o manipulación equivocada de documentos físicos puede ocasionar la pérdida total de datos importantes. Otra consecuencia significativa es la tardanza en procesar información, pues al no existir un sistema automatizado, cada consulta o actualización requiere tiempo adicional y depende de la disponibilidad de los registros manuales.

Por otro lado, muchos negocios de reciente creación buscan soluciones accesibles, económicas y fáciles de implementar, pero carecen de alternativas que no exijan infraestructura costosa o compleja. Este escenario evidencia la necesidad de un software sencillo, confiable y diseñado específicamente para cubrir las necesidades básicas de control de inventarios sin requerir una curva de aprendizaje elevada. De igual manera, es indispensable que dicho sistema permita generar archivos digitales en formatos accesibles, como .txt, para almacenar los datos de manera organizada y asegurar la transparencia, permanencia y portabilidad de la información. En este contexto surge la propuesta de desarrollar un sistema de gestión de inventarios programado en C++, el cual permita a las empresas pequeñas administrar sus productos de forma clara, eficiente y segura. Este software busca atender directamente la problemática mencionada, proporcionando una herramienta capaz de simplificar los procesos de registro, consulta, modificación y eliminación de datos, con el fin de mejorar la precisión y disminuir los errores humanos. Al integrarse con archivos .txt, el sistema ofrece una alternativa ligera y portable que puede funcionar sin necesidad de bases de datos avanzadas o servidores dedicados, lo que facilita su uso en equipos de cómputo convencionales.

La problemática actual demuestra que la falta de herramientas accesibles y bien diseñadas representa una barrera importante para la correcta administración del inventario en empresas emergentes. Con un sistema desarrollado específicamente para este contexto, se busca aportar una solución funcional que permita a los usuarios mantener información confiable, mejorar la organización interna y agilizar los procesos operativos relacionados con la gestión del inventario.

Objetivos

El presente proyecto tiene como propósito establecer una solución tecnológica accesible, eficiente y adaptable para la gestión de inventarios dentro de empresas pequeñas o en etapa inicial. Debido a que estas organizaciones suelen enfrentar limitaciones económicas, de tiempo y de personal capacitado, el desarrollo de un software en lenguaje C++ que funcione mediante archivos de texto (.txt) representa una alternativa viable y práctica para mejorar la administración de sus productos. El objetivo general del proyecto es diseñar e implementar un sistema que permita registrar, consultar, actualizar y eliminar información relacionada con el inventario de manera ordenada, precisa y fácil de utilizar, garantizando que cualquier usuario, incluso sin experiencia previa en herramientas digitales avanzadas, pueda comprender su funcionamiento sin dificultad.

Dentro de este marco, uno de los objetivos específicos consiste en estructurar una interfaz de uso simple que reduzca la curva de aprendizaje y facilite la interacción con el sistema. Esto implica diseñar menús claros, opciones bien delimitadas y mensajes que orienten al usuario durante cada operación. Asimismo, se busca generar un mecanismo confiable de almacenamiento mediante archivos .txt, con el fin de que la información registrada se mantenga accesible, portable y segura, sin depender de bases de datos complejas ni requerir configuraciones técnicas especializadas. Otro objetivo importante es garantizar que el sistema procese adecuadamente las operaciones de entrada y salida de inventario, evitando inconsistencias mediante validaciones que permitan identificar errores, datos faltantes o valores inválidos antes de guardar los cambios.

Además, el proyecto tiene como objetivo desarrollar una estructura modular de programación que permita futuras ampliaciones o mejoras del sistema sin necesidad de modificarlo por completo. Esta característica es fundamental para asegurar que el software pueda adaptarse a las necesidades cambiantes de las empresas, ya sea incorporando etiquetas adicionales, categorías de productos, reportes automatizados o nuevas funciones dentro del archivo de inventario. De igual manera, se busca implementar un mecanismo que permita al usuario consultar el historial o estado actual del inventario de forma rápida y ordenada, reduciendo el tiempo que normalmente se invierte en revisar documentos manuales o archivos desorganizados.

Otro objetivo específico es contribuir a la disminución de errores humanos durante el proceso de registro de datos. Esto se logrará mediante mensajes de confirmación, validaciones de datos y procedimientos que soliciten información estricta y estructurada antes de aceptar cualquier movimiento. De esta forma, el sistema no solo se limita a almacenar datos, sino que ofrece un apoyo real para mejorar la calidad de la información generada por los usuarios. Finalmente, el proyecto busca fomentar el uso de herramientas tecnológicas libres y de fácil acceso, demostrando que es posible crear soluciones funcionales y profesionalmente útiles mediante lenguajes de programación clásicos como C++, sin depender de plataformas costosas o software propietario.

En conjunto, todos estos objetivos forman la base para la creación de un sistema de inventarios sólido, confiable y adaptable, orientado a resolver problemas reales dentro del entorno operativo de pequeñas empresas. Con este proyecto se pretende no solo cubrir una necesidad inmediata, sino también establecer un modelo de organización y gestión que pueda expandirse en el futuro conforme las necesidades de la empresa evolucionen.

Justificación

La realización de este proyecto de Sistema de Gestión de Inventarios (SGI) se justifica plenamente como una respuesta crítica a la inaccesibilidad tecnológica que limita el crecimiento de las Micro y Pequeñas Empresas (MiPyMEs). El problema fundamental radica en que las soluciones comerciales de software avanzado son inalcanzables para este sector, imponiendo costos elevados y requisitos de infraestructura compleja (servidores y bases de datos robustas). Esta exclusión fuerza a las MiPyMEs a depender de métodos de control manuales (hojas de cálculo o registros físicos).

¿Por qué se va a realizar el proyecto?

El proyecto se va a realizar para mitigar las graves consecuencias operativas de la gestión manual. La dependencia de estos métodos provoca una exposición constante al error humano, lo que se traduce directamente en pérdidas económicas por sobreinventario (inmovilización de capital) o por desabasto inesperado (pérdida de ventas). Además, la falta de un sistema estructurado genera datos inconsistentes y la vulnerabilidad de la información ante el extravío o la manipulación, impidiendo a la gerencia tomar decisiones informadas.

Este SGI está diseñado para ser la solución accesible que el sector necesita. Al desarrollarse en C++, se garantiza una aplicación eficiente, rápida y ligera. La clave de su justificación reside en el uso de archivos de texto (.txt) para el almacenamiento. Esto elimina la necesidad de bases de datos caras y complejas, reduciendo el costo total de propiedad (TCO) y facilitando su uso en cualquier equipo de cómputo convencional. El proyecto busca democratizar la gestión digital, proveyendo una herramienta que automatice las operaciones esenciales (registro, consulta, etc.) y asegure la precisión de los datos.

Área de Impacto

El impacto del proyecto es doble:

1. **Impacto Local:** Beneficio directo a las MiPyMEs regionales, al reducir sus pérdidas operativas y mejorar su productividad y organización interna. Al dotarles de información confiable, se fortalece el tejido empresarial local.
2. **Impacto Global:** El modelo de solución establece un precedente viable de software de peso ligero (C++ y .txt) para la gestión empresarial. Esto lo convierte en un prototipo fácilmente replicable en otras economías en desarrollo o regiones con restricciones presupuestarias o de infraestructura, contribuyendo a la inclusión digital a escala global.

En resumen, el proyecto se justifica como una inversión estratégica en la eficiencia, la accesibilidad y el crecimiento sostenible de las empresas emergentes.

Estado del arte

El manejo de inventarios es una función crítica en cualquier comercio: permite conocer existencias, evitar rupturas de stock o exceso de inventario, optimizar costos y mejorar servicio al cliente. En las últimas dos décadas los sistemas de inventario han evolucionado desde hojas manuales y módulos locales en ERPs hacia soluciones en la nube, movilidad (apps móviles/pos), integración con dispositivos IoT (sensores, lectores RFID) y uso de técnicas de análisis predictivo (machine learning) para pronóstico de demanda y optimización. Esta revisión presenta antecedentes teóricos y prácticos, tecnologías actuales, comparativa entre enfoques y recomendaciones para un proyecto de software de inventario.

Antecedentes y fundamentos teóricos

Históricamente, la gestión de inventarios parte de modelos clásicos de la investigación operativa:

- EOQ (Economic Order Quantity): modelo para calcular la cantidad óptima de pedido que minimiza costos de pedido y de mantenimiento.
- JIT (Just-In-Time): filosofía para reducir existencias manteniendo entregas frecuentes y sincronizadas con demanda.
- ABC analysis: clasificación de productos según valor o movimiento (A = pocos ítems de alto valor, B = intermedios, C = muchos de bajo valor), para priorizar control y recursos.

Estos conceptos permanecen vigentes y suelen combinarse con herramientas automatizadas: por ejemplo, EOQ y ABC siguen siendo bases para políticas de reabastecimiento que luego se implementan en software; JIT motiva integraciones con proveedores y sistemas de pedidos automáticos. La literatura académica reciente sintetiza estos enfoques y su integración con tecnologías digitales modernas.

Tecnologías, términos y tendencias actuales

1. Códigos de barras y lectores: solución estándar, barata y ampliamente adoptada para registro de entradas/salidas.
2. RFID + IoT: permite lectura sin línea de vista, conteo automático y monitorización en tiempo real; se emplea en almacenes y retail cuando se necesita mayor trazabilidad. Integrar RFID con una arquitectura IoT (sensores, gateways, nube) aumenta precisión y reduce inventarios ciegos.
3. Sistemas en la nube y POS móviles: proliferan soluciones SaaS que sincronizan ventas multicanal (tienda física, e-commerce) y permiten gestión centralizada y multi-localidad. Las ventajas son actualizaciones automáticas, escalabilidad y menores costos iniciales.
4. Inteligencia artificial y ML: se usan para pronóstico de demanda, detección de anomalías, y optimización de inventario dinámico. Estudios recientes muestran mejoras en precisión de pronóstico y reducción de costos al integrar ML con reglas tradicionales.

5. Integraciones y APIs: la interoperabilidad (ERP, e-commerce, contabilidad, POS) es determinante. Muchas implementaciones reales combinan módulos especializados con integradores o middleware que mantienen coherencia de stock.

Revisión de literatura/estudios representativos

- Revisiones académicas y artículos técnicos muestran una evolución clara: desde soluciones locales orientadas a procesos manuales hasta propuestas IoT + ML que automatizan conteo y mejoran pronóstico. Un artículo de revisión (2024) sintetiza esta transición y discute beneficios (reducción de faltantes, mejora en precisión) y desafíos (costos de implementación, seguridad de datos).
- Trabajos propositivos y tesis (2024–2025) describen prototipos que integran sensores IoT, RFID y algoritmos de ML para pequeñas bodegas, con resultados preliminares prometedores en tiempo real y reducción de inventario muerto. Esto refleja una tendencia académica hacia soluciones híbridas prácticas.

Comparativa entre trabajos y sistemas similares

Open source / soluciones comunitarias (ej.: ERPNext, Odoo)

- ERPNext: integrado, buena cobertura de inventario, control de lotes, multi-ubicación; fuerte en comunidades técnicas y fácil de personalizar para PYMEs.
- Odoo: muy modular y con amplia oferta de apps; fuerte en UX y marketplace de módulos, pero algunas críticas sobre profundidad funcional en ciertos módulos de inventario en su versión comunitaria.

Comparativas recientes (2024–2025) muestran que la elección entre Odoo y ERPNext depende de necesidades: customización vs profundidad de módulos empresariales; en general, Odoo tiende a ofrecer mejor experiencia de usuario y ecosistema comercial, ERPNext se percibe como más “completo” en funcionalidades nativas para inventario en ciertas implementaciones.

Soluciones comerciales (ej.: SAP Business One, QuickBooks, NetSuite)

- SAP Business One / NetSuite: orientadas a empresas en crecimiento/pleno crecimiento; ofrecen integración ERP completa (finanzas, compras, MRP) y buenas capacidades para gestión de inventarios complejos.

- QuickBooks: más contabilidad con módulos de inventario limitados; adecuado para pequeñas tiendas con necesidad contable básica pero no para procesos logísticos complejos.

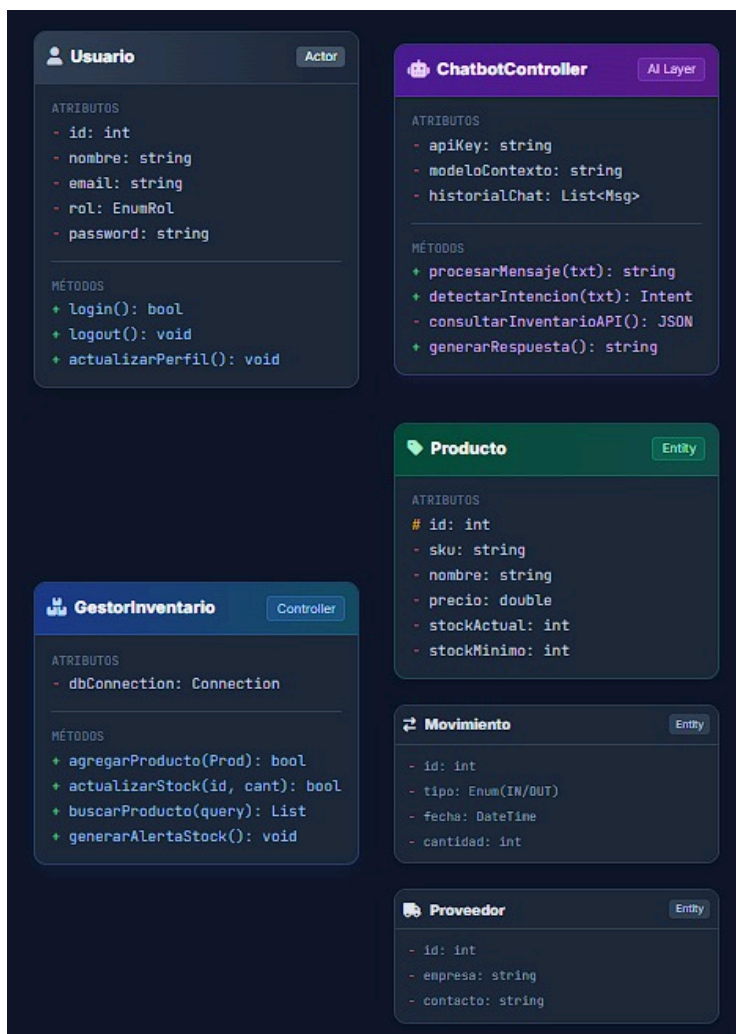
Comparaciones de mercado sugieren que SAP y ERPs grandes son mejores cuando hay procesos de manufactura, MRP o multi-localidad compleja; QuickBooks y soluciones SaaS más pequeñas son más rápidos de desplegar y económicos para comercios que no requieren funcionalidades avanzadas.

Hipótesis

Con este trabajo se pretende demostrar que la integración de un chatbot en un sistema de gestión de inventarios optimiza significativamente la eficiencia operativa, reduciendo el tiempo empleado en consultas de stock y registro de movimientos en comparación con las interfaces gráficas tradicionales. Asimismo, se plantea que el uso de lenguaje natural para la interacción con la base de datos disminuirá la curva de aprendizaje para nuevos usuarios y minimizará los errores humanos asociados a la captura manual de datos, garantizando una mayor integridad en la información del inventario.

Diseño

Diagrama de clases



Requerimientos Funcionales

El sistema de gestión de inventarios desarrollado en C++ deberá cumplir con un conjunto de requerimientos funcionales que garanticen su correcto desempeño y la utilidad real para las empresas pequeñas que necesitan administrar sus productos sin depender de bases de datos complejas. En primer lugar, el sistema debe permitir registrar nuevos productos mediante un proceso de alta que solicite al usuario información esencial como el identificador único, nombre, categoría, cantidad inicial y precio unitario. Esta información se almacenará de manera permanente en un archivo de texto, asegurando la persistencia de los datos incluso después de cerrar el programa.

Otro requerimiento fundamental es la capacidad de consultar productos registrados en el inventario. El usuario deberá tener la posibilidad de realizar búsquedas por ID, nombre o categoría, obteniendo como resultado los datos completos del producto. Esto permitirá una verificación rápida del estatus de cada artículo, su disponibilidad y cualquier otra característica relevante. La consulta debe realizarse de forma eficiente, utilizando estructuras internas como vectores para cargar los registros en memoria temporal, con el objetivo de acelerar el procesamiento mientras el programa está en ejecución.

Asimismo, el sistema deberá permitir modificar la información de un producto. Esto implica editar valores como nombre, cantidad, categoría o precio. El requerimiento funcional establece que la modificación debe ejecutarse únicamente si el ID ingresado pertenece a un producto existente, y que la actualización debe reflejarse tanto en la memoria interna como en el archivo .txt correspondiente. Del mismo modo, el sistema debe ofrecer la opción de dar de baja un producto, eliminándolo del registro almacenado. Esta operación debe reescribir la estructura del archivo para asegurar que la información quede ordenada y sin inconsistencias.

El control de entradas y salidas del inventario constituye otro requerimiento esencial. El sistema deberá registrar movimientos de mercancía permitiendo sumar o restar unidades según se trate de una reposición o una venta. Cada movimiento debe quedar guardado en un archivo de historial, indicando fecha, hora, producto afectado, cantidad, tipo de movimiento y usuario responsable en caso de que el módulo de usuarios esté habilitado. Esto permitirá mantener un control detallado de todas las acciones realizadas dentro del inventario, facilitando auditorías internas o revisiones posteriores.

Además, el sistema debe ser capaz de cargar automáticamente los datos desde los archivos .txt al iniciar la ejecución, validando la estructura del archivo y evitando errores si el archivo está dañado o vacío. En caso de que el archivo no exista, el sistema deberá crearlo automáticamente sin interrumpir la funcionalidad. También deberá incluir opciones de menú claras para que el usuario pueda navegar entre las operaciones de alta, baja, consulta, modificación y movimientos de manera intuitiva. La navegación debe ser secuencial y guiada por mensajes, de forma que incluso los usuarios sin experiencia en software puedan interactuar con el sistema sin dificultades.

Finalmente, el sistema deberá manejar errores comunes, como valores no numéricos, IDs inexistentes, cantidades negativas o intentos de eliminar o modificar productos inexistentes. Estas validaciones son necesarias para garantizar la integridad de los datos, evitar fallas en la ejecución y asegurar que el inventario refleje la realidad operativa de la empresa. Con estos requerimientos funcionales, el software cumple con los elementos necesarios para brindar una herramienta confiable, sencilla y práctica para la administración de inventarios basada en archivos de texto.

Requerimientos No Funcionales

Los requerimientos no funcionales del sistema de gestión de inventarios determinan las características de calidad que debe cumplir el software para garantizar un funcionamiento confiable, eficiente y adecuado a las necesidades de empresas pequeñas. En primer lugar, el sistema debe ser portátil, de modo que pueda ejecutarse en distintos equipos con sistemas Windows sin necesidad de instalaciones adicionales ni librerías externas. Este requerimiento implica que todo el proyecto esté programado exclusivamente en C++ estándar, utilizando únicamente funciones básicas de manejo de archivos y lectura de consola, lo que garantiza su compatibilidad con cualquier entorno común.

En términos de rendimiento, el sistema debe ser capaz de cargar, procesar y mostrar la información del inventario de manera rápida, aun si el archivo .txt contiene una lista extensa de productos. Para ello, el software hará uso de estructuras internas en memoria, como vectores, que permiten acceder y recorrer los datos de forma eficiente. Se debe procurar que las operaciones más frecuentes —consultas, altas, bajas y modificaciones— se ejecuten en un tiempo menor al perceptible por el usuario promedio. Esto permite que el sistema sea suficientemente rápido incluso en computadoras de bajo rendimiento, como las que comúnmente utilizan las microempresas o negocios que están empezando.

Otro requerimiento no funcional importante es la usabilidad. El sistema debe presentar menús claros, textos comprensibles y un flujo de navegación que evite confusiones. Cada opción debe estar acompañada de mensajes que indiquen exactamente qué debe ingresar el usuario y cuál es el siguiente paso dentro del proceso. Este aspecto es especialmente relevante considerando que el sistema está dirigido a usuarios sin conocimientos técnicos avanzados. La interfaz debe estar diseñada para reducir la curva de aprendizaje, minimizar errores durante la captura de datos y permitir que cualquier persona pueda usarlo en pocos minutos sin necesidad de capacitación formal.

En cuanto a la seguridad, si bien el sistema no utiliza una base de datos compleja, debe asegurar la integridad de la información almacenada en los archivos .txt. Para cumplir con este requerimiento, el software deberá incluir validaciones estrictas que eviten que valores incorrectos o datos incompletos sean guardados en los archivos. El sistema también debe evitar que se pierda información por errores del usuario, por lo que deberá solicitar confirmaciones antes de operaciones irreversibles, como la eliminación de un registro. Cuando se manejan movimientos de inventario, el sistema debe generar un historial que permita reconstruir los cambios realizados y detectar posibles inconsistencias.

La mantenibilidad es otro aspecto clave. El código del proyecto debe estar organizado en funciones o clases bien separadas, con nombres significativos y sin dependencias innecesarias. Esto permitirá que el sistema pueda actualizarse fácilmente en el futuro, ya sea para agregar nuevos módulos, cambiar la estructura del archivo o integrar funciones más avanzadas. La modularidad del sistema facilita la depuración, la ampliación de capacidades y la adaptación del software a distintos escenarios empresariales sin tener que reescribir grandes partes del código.

Por último, el sistema debe cumplir con el requerimiento no funcional de confiabilidad. Todas las operaciones deben ejecutarse sin fallos, y en caso de que ocurran errores de entrada o situaciones inesperadas, el sistema deberá responder de manera controlada, nunca cerrándose abruptamente ni permitiendo que los archivos queden dañados. El manejo adecuado de excepciones mediante validaciones internas es fundamental para garantizar que el inventario siempre se mantenga correcto y que los usuarios puedan confiar plenamente en el sistema para tomar decisiones operativas. En conjunto, estos requerimientos no funcionales aseguran que el software no solo cumpla con su propósito, sino que lo haga con calidad, estabilidad y facilidad de uso.

Casos de Uso

Caso de uso 1: Registrar producto (Alta)

Actor: Usuario del sistema.

Propósito: Agregar un nuevo producto al inventario.

Precondición: El archivo productos.txt debe existir o el sistema debe poder crearlo automáticamente.

Flujo básico:

1. El usuario selecciona la opción de “Registrar producto” desde el menú principal.
2. El sistema solicita los datos del producto: ID, nombre, categoría, cantidad y precio.
3. El usuario ingresa toda la información requerida.
4. El sistema valida que el ID no exista previamente y que los valores sean correctos.
5. El sistema agrega el producto al vector interno y actualiza el archivo productos.txt.

Postcondición: El producto queda disponible para búsquedas, modificaciones y movimientos.

Caso de uso 2: Consultar producto

Actor: Usuario del sistema.

Propósito: Obtener la información de uno o varios productos.

Precondición: El archivo debe contener al menos un registro válido.

Flujo básico:

1. El usuario selecciona “Consultar producto” desde el menú.
2. El sistema ofrece opciones de búsqueda: por ID, nombre o categoría.
3. El usuario elige el método y proporciona el dato correspondiente.

4. El sistema realiza la consulta en el vector de productos.
5. Se muestra la información completa del producto o lista de coincidencias.

Postcondición: El usuario visualiza información sin afectar el inventario.

Caso de uso 3: Modificar producto

Actor: Usuario del sistema.

Propósito: Cambiar los datos de un producto existente.

Precondición: El ID ingresado debe existir.

Flujo básico:

1. El usuario selecciona “Modificar producto”.
2. Ingresa el ID del producto que desea editar.
3. El sistema valida si existe el producto.
4. El usuario ingresa los nuevos valores.
5. El sistema actualiza el vector y sobrescribe productos.txt.

Postcondición: El producto queda actualizado con los nuevos valores.

Caso de uso 4: Eliminar producto (Baja)

Actor: Usuario del sistema.

Propósito: Eliminar definitivamente un registro del inventario.

Precondición: El ID del producto debe existir.

Flujo básico:

1. El usuario selecciona “Eliminar producto”.
2. El sistema solicita el ID.
3. Se valida que el ID exista.
4. El sistema muestra un mensaje de confirmación.
5. El usuario confirma la eliminación.
6. El sistema elimina el producto del vector y reescribe el archivo sin ese registro.

Postcondición: El producto deja de estar disponible dentro del sistema.

Caso de uso 5: Registrar movimiento (Entrada/Salida)

Actor: Usuario del sistema.

Propósito: Registrar cambios en la cantidad de un producto.

Precondición: Debe existir el producto.

Flujo básico:

1. El usuario selecciona “Registrar movimiento”.
2. Indica si se trata de una entrada o una salida.
3. Ingresa el ID del producto y la cantidad.
4. El sistema valida existencia y cantidad disponible (si es salida).
5. Actualiza la cantidad del producto.
6. Registra la transacción en movimientos.txt con fecha y hora.

Postcondición: La cantidad del inventario se modifica correctamente.

Caso de uso 6: Cargar archivos al iniciar

Actor: Sistema.

Propósito: Cargar información almacenada en archivos .txt.

Precondición: Los archivos deben existir o ser creados automáticamente.

Flujo básico:

1. El sistema detecta si existen archivos de productos y movimientos.
2. Si no existen, crea archivos vacíos.
3. Si existen, lee cada línea y reconstruye los objetos Producto y Transacción.
4. Llena los vectores internos para uso durante la sesión.

Postcondición: El sistema se encuentra listo para operar.

Caso de uso 7: Guardar cambios antes de salir

Actor: Sistema.

Propósito: Asegurar la persistencia de todos los datos.

Precondición: Haber realizado modificaciones durante la sesión.

Flujo básico:

1. El usuario selecciona “Salir”.
2. El sistema verifica si hay cambios pendientes.
3. Si los hay, sobrescribe los archivos .txt con la información actualizada.
4. Cierra el programa de forma segura.

Postcondición: Los datos quedan almacenados de manera permanente.

Arquitectura del Sistema

La arquitectura del sistema del gestor de inventario está diseñada de manera modular para permitir que cada componente cumpla una función específica y pueda mantenerse o modificarse sin afectar al resto del software. Aunque normalmente esta sección se mostraría mediante un esquema visual, en este documento se describe de manera completamente textual, manteniendo el nivel de detalle necesario para entender la organización interna del proyecto. La arquitectura está basada en tres capas principales: la capa de presentación, la capa de lógica de negocio y la capa de persistencia, cada una con responsabilidades claramente definidas.

La capa de presentación corresponde a la interfaz textual con la que interactúa el usuario. Esta capa maneja los menús, las opciones y los mensajes que guían al operador durante el proceso de registrar, eliminar, buscar o consultar productos dentro del inventario. Aunque es una interfaz simple basada en consola, se encuentra organizada para ser intuitiva, clara y libre de ambigüedades. Cada sección del menú está diseñada para solicitar únicamente los datos necesarios, validarlos e informar al usuario sobre el resultado de cada operación. Esta capa no realiza cálculos complejos ni manipulación directa de los archivos; únicamente recibe datos, los envía a la lógica de negocio y muestra los resultados que esta retorna.

La capa de lógica de negocio es el núcleo del sistema. Aquí se encuentran las reglas que controlan cómo debe funcionar el inventario. Esta capa está representada principalmente por la clase *Inventario*, que administra los productos registrados a través de métodos bien definidos. Entre estas operaciones se encuentran agregar un producto, eliminarlo, modificarlo, realizar búsquedas, calcular totales y generar listados. También se incluye la validación de datos, como verificar que no se registren códigos duplicados o que las cantidades y precios sean correctos. Dentro de esta misma capa se encuentra la clase *Producto*, la cual define la estructura fundamental de cada artículo y permite encapsular atributos como nombre, código, cantidad y precio. La lógica de negocio cumple con el principio de responsabilidad única: únicamente se dedica a procesar la información y garantizar que se mantengan las reglas del inventario.

La tercera capa, la capa de persistencia, está encargada de almacenar la información de manera permanente. Esta responsabilidad recae en la clase GestorArchivo, que se encarga de leer y escribir los datos en un archivo con extensión .txt. Su arquitectura está diseñada para separar completamente el almacenamiento del resto del sistema, de modo que el Inventario solo se ocupa de manejar datos en memoria, mientras que el GestorArchivo se especializa en sincronizar dichos datos con el archivo físico. Esta separación permite que el sistema sea más fácil de mantener y ofrece la posibilidad de reemplazar la forma de almacenamiento en el futuro sin modificar la lógica principal. Por ejemplo, si en algún momento se quisiera migrar a archivos binarios o incluso a una base de datos, solo sería necesario cambiar esta capa, manteniendo intacto el funcionamiento del inventario.

La interacción entre las tres capas sigue un flujo bien estructurado: el usuario opera desde la capa de presentación, que envía solicitudes a la capa de lógica de negocio; esta ejecuta la operación, valida los datos y actualiza la información en memoria; posteriormente, la capa de persistencia guarda los cambios en el archivo correspondiente. Este flujo evita dependencias innecesarias y reduce la complejidad general del programa, permitiendo que cada componente pueda probarse o depurarse de manera independiente.

En conjunto, esta arquitectura modular logra que el sistema sea robusto, escalable y fácil de extender. La organización en capas también mejora la claridad del código y facilita el trabajo colaborativo, ya que cada integrante del equipo puede trabajar en una sección diferente sin interferir con las demás. Esta estructura representa una base sólida para el desarrollo de un gestor de inventarios funcional, bien organizado y apto para ser utilizado por empresas pequeñas que requieren un control básico pero confiable de su mercancía.

Manual de Usuario

El manual de usuario del sistema de gestión de inventarios tiene como propósito orientar al operador en el uso correcto del software, explicando de manera clara cada una de las funciones disponibles dentro del menú principal. Este documento está diseñado para usuarios sin experiencia técnica previa, utilizando descripciones simples y directas que permitan comprender cómo deben ingresarse los datos y cómo interpretar los mensajes que muestra el programa. El sistema funciona completamente en consola, por lo que todas las interacciones se realizan mediante texto y selección numérica de opciones.

Al iniciar el programa, el usuario visualizará el menú principal, el cual contiene todas las funciones esenciales para la administración del inventario. Cada opción del menú está numerada del 1 al 6, y el usuario solo debe escribir el número correspondiente para acceder a la operación deseada. Las opciones incluyen registrar un producto nuevo, consultar la información de un artículo existente, modificar registros, eliminar productos, generar un listado general y salir del sistema. El menú está diseñado para mantenerse visible después de cada operación realizada, permitiendo que el usuario continúe trabajando sin necesidad de reiniciar el programa.

Para registrar un producto, el usuario debe seleccionar la opción “Agregar producto”. El sistema solicitará ingresar la información necesaria: nombre del producto, código único, cantidad disponible y precio unitario. Cada dato debe escribirse cuidadosamente para evitar errores. El software incluye validaciones que impiden registrar códigos duplicados, cantidades negativas o precios inválidos. Si algún dato no cumple con los criterios aceptados, se mostrará un mensaje de advertencia indicando el error y el usuario podrá corregir la información antes de continuar. Una vez que los datos son correctos, el sistema confirma la creación del producto y lo almacena tanto en memoria como en el archivo .txt correspondiente.

Para consultar un producto, el usuario debe seleccionar la opción “Buscar producto”. El sistema solicitará el código del artículo que desea localizar. Si el producto existe dentro del inventario, se mostrará toda su información de manera detallada. En caso de que no exista, el sistema mostrará un mensaje indicando que no se encontró ningún registro con ese código, permitiendo al usuario intentar nuevamente o regresar al menú principal.

Para modificar un producto existente, el usuario debe seleccionar la opción “Modificar producto” dentro del menú principal. El sistema solicitará ingresar el código del artículo que se desea actualizar. En caso de que el código exista, se mostrará la información actual del producto y el sistema permitirá elegir qué atributo se desea cambiar: nombre, cantidad o precio. Cada modificación se realiza de manera independiente, y el sistema pedirá confirmar el cambio antes de aplicarlo. Si el usuario introduce un valor no válido, como cantidades negativas o un precio incorrecto, el sistema mostrará un mensaje de advertencia y no permitirá continuar hasta que el dato sea corregido. Una vez confirmada la modificación, el sistema actualizará automáticamente tanto la información en memoria como en el archivo .txt para mantener la integridad del inventario.

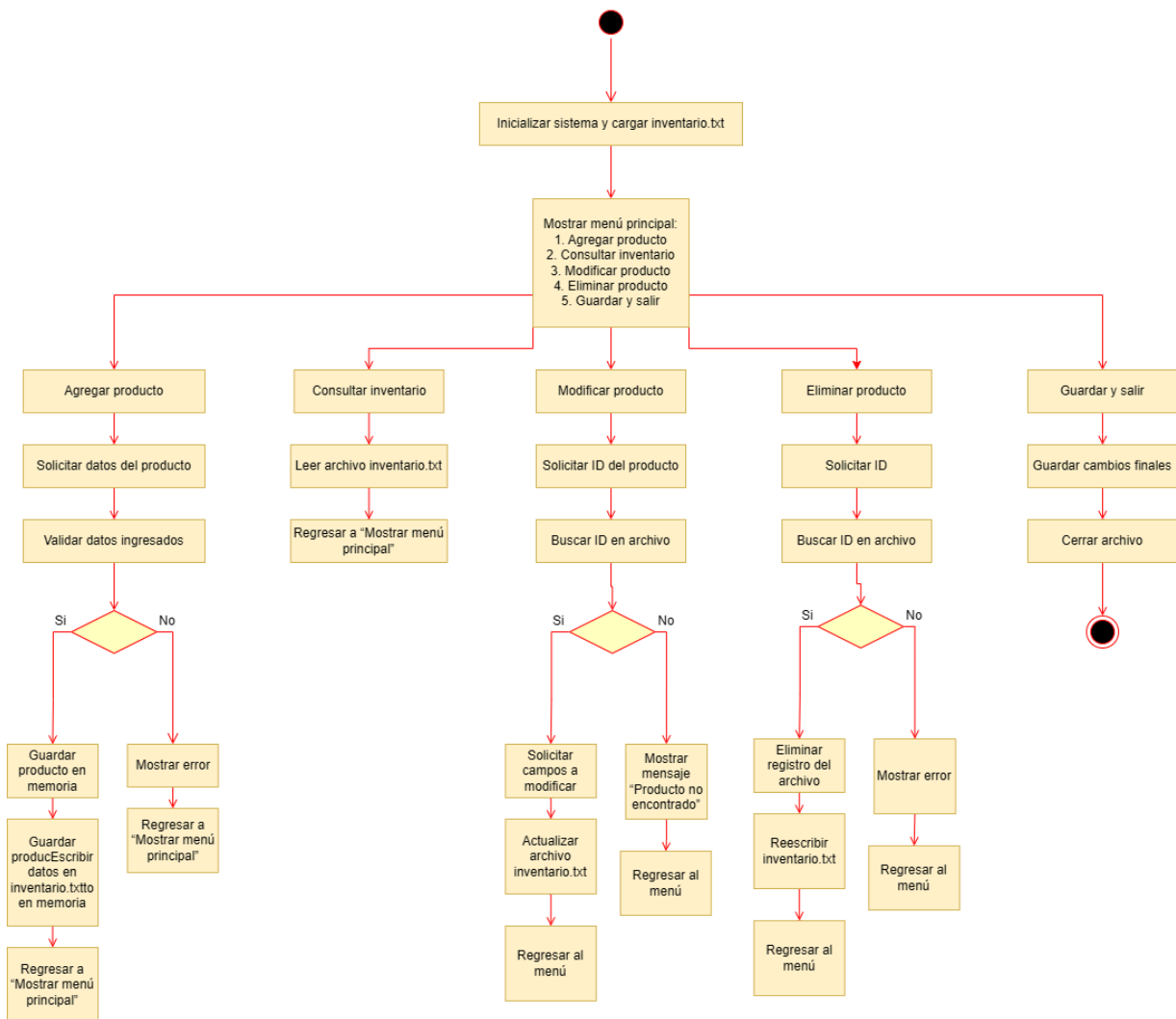
La opción de eliminar un producto permite retirar definitivamente un artículo del inventario. Para ello, el usuario debe seleccionar “Eliminar producto” y escribir el código del producto que desea borrar. El sistema buscará dicho código y, si lo encuentra, mostrará los datos del artículo para que el usuario pueda verificar que se trata del registro correcto. Posteriormente, solicitará una confirmación explícita, ya que esta acción es irreversible. Si el usuario confirma, el producto será eliminado de la lista en memoria y el archivo será reescrito sin dicho registro. Esta funcionalidad está diseñada para evitar pérdidas accidentales de información y garantizar que solo se eliminen productos con total certeza.

La opción “Mostrar inventario completo” genera un listado organizado de todos los productos registrados en el sistema. Al seleccionar esta acción, el usuario podrá ver en pantalla cada producto con su código, nombre, cantidad y precio. Esta función es útil para obtener una visión general del inventario actualizado. Debido a que el sistema trabaja con archivos .txt, el listado siempre refleja la información real contenida en la base de datos del proyecto. En caso de que no existan productos registrados, se mostrará un mensaje indicando que el inventario está vacío.

Finalmente, dentro del menú principal se encuentra la opción “Salir del sistema”, la cual permite cerrar el programa de manera segura. Al seleccionarla, el sistema verifica que toda la información en memoria haya sido correctamente almacenada en el archivo antes de finalizar la ejecución. Esta medida evita pérdidas de datos y garantiza que el inventario quede actualizado para la próxima vez que se utilice el software.

El manejo del archivo .txt es completamente automático para el usuario; el sistema realiza la lectura y escritura del archivo sin que sea necesario que el operador manipule directamente los datos. Esto permite que personas sin experiencia técnica puedan utilizar el software de forma segura y eficiente. En su conjunto, el manual de usuario proporciona las indicaciones necesarias para asegurar que cualquier persona pueda registrar, consultar, actualizar y eliminar productos sin dificultad, permitiendo que la administración del inventario sea un proceso accesible y confiable.

Diagrama de Actividad



Implementación

El desarrollo del software de inventario para un comercio requirió un proceso de trabajo estructurado, dividido en etapas claramente definidas que permitieron avanzar desde la concepción del problema hasta la implementación de un sistema funcional. A continuación se detalla cómo se trabajó durante la construcción del proyecto, describiendo las tareas realizadas, las herramientas seleccionadas y las razones técnicas y prácticas que guiaron cada decisión.

1. Análisis del problema y levantamiento de requerimientos

El primer paso consistió en comprender el contexto real del comercio y la manera en que actualmente se gestionaban los productos. Se realizaron entrevistas breves con el responsable del negocio para identificar fallas comunes: pérdida de información, falta de control sobre las existencias, errores en conteos manuales, duplicidad de registros y ausencia de reportes claros de ventas y movimientos.

A partir de este diagnóstico se elaboró una lista inicial de requerimientos funcionales y no funcionales. Los principales requerimientos funcionales incluyeron: registrar productos, añadir y retirar existencias, consultar niveles de inventario, editar información de artículos, generar reportes básicos y, opcionalmente, integrar un punto de venta simple. En cuanto a requerimientos no funcionales, se definió que el sistema debía ser fácil de usar, seguro, con tiempos de respuesta adecuados, escalable para añadir módulos futuros y compatible con equipos de bajo consumo.

Esta primera fase permitió tener una visión completa del problema y justificar la construcción de un sistema que automatiza y centraliza el inventario del comercio.

2. Diseño de la arquitectura del sistema

La siguiente etapa consistió en definir la estructura general del sistema. Se optó por una arquitectura modular que permitiera dividir el proyecto en componentes independientes, facilitando el mantenimiento y la posibilidad de ampliar funcionalidades en el futuro.

Los módulos contemplados fueron:

- Gestor de productos: encargado del almacenamiento, modificación y consulta de datos de cada artículo.
- Control de inventario: responsable de entradas, salidas y ajustes de stock.

- Módulo de reportes: generador de informes sobre existencias, productos con bajo stock y movimientos realizados.
- Interfaz de usuario: capa visual que permite operar el sistema de forma intuitiva.

Al organizar el proyecto en módulos fue posible asignar tareas específicas a cada integrante del equipo, evitando confusiones y promoviendo el trabajo simultáneo en distintos componentes sin generar conflictos.

3. Elección de herramientas y lenguajes de programación

Para el desarrollo se evaluaron varias alternativas y finalmente se eligieron herramientas basadas en criterios de accesibilidad, facilidad de implementación y compatibilidad con el entorno académico y del comercio.

Lenguaje de programación

Se utilizó C++ debido a varias razones:

- Permite un control detallado del uso de memoria, algo importante al trabajar con estructuras de datos como vectores, listas enlazadas y estructuras personalizadas.
- Facilita la creación de clases y módulos, lo cual se adapta bien a la estructura del proyecto.
- Es un lenguaje ampliamente enseñado en cursos de programación, por lo que era accesible para todos los integrantes del equipo.

Además, C++ proporciona un rendimiento adecuado para sistemas donde se realizan búsquedas, modificaciones y consultas constantes sobre la base de datos local del inventario.

Entorno de desarrollo

El proyecto se trabajó principalmente en Visual Studio Code debido a su ligereza, extensiones de soporte para C++, depuración integrada y facilidad para gestionar carpetas y archivos del proyecto.

Sistema de almacenamiento

En la primera versión del sistema se decidió utilizar archivos de texto o archivos binarios para almacenar la información del inventario. Esta elección se basó en la necesidad de un sistema sencillo, funcional y fácil de transportar sin requerir un servidor de base de datos completo. Más adelante, el diseño modular deja abierta la posibilidad de migrar a SQLite o MySQL si el comercio necesita una versión más robusta.

Control de versiones

Aunque no se utilizó un repositorio formal en línea, se aplicaron prácticas básicas de control de versiones mediante copias organizadas y comentarios en el código para llevar registro de cambios, agregados y correcciones en el sistema a lo largo del desarrollo.

4. Desarrollo de la interfaz de usuario

La interfaz se diseñó con el objetivo de ser clara, minimalista y fácil de usar para usuarios sin experiencia técnica. Se realizó un prototipo de las pantallas principales: menú, registro de productos, consulta y reportes.

Se evitó el exceso de elementos visuales para no complicar la navegación y se estructuraron las opciones de forma jerárquica. También se cuidó que los mensajes del sistema fueran entendibles, por ejemplo: “Producto registrado correctamente” o “Existencias insuficientes para realizar la operación”.

5. Implementación de las funcionalidades principales

El desarrollo de funciones se llevó a cabo de forma incremental. Se comenzó por las funciones esenciales del inventario:

1. Agregar productos: creación de objetos con atributos como ID, nombre, cantidad y precio.
2. Buscar productos: mediante algoritmos de búsqueda lineal, adecuados para un inventario pequeño y con posible evolución a búsqueda binaria o estructuras más avanzadas.
3. Modificar productos: actualización de datos cuando existe un cambio en precio, descripción o cantidad.
4. Entradas y salidas de inventario: controles que permiten llevar un seguimiento preciso de los movimientos.
5. Reportes: generación de listados de productos, notificaciones de bajo stock y movimientos realizados.

Cada una de estas funciones se probó individualmente antes de integrarse al sistema completo, reduciendo errores y permitiendo avances constantes.

6. Pruebas y validación del sistema

Con las funciones implementadas, se inició la etapa de pruebas. Se simularon situaciones reales del comercio, como registrar nuevos productos, realizar ventas, corregir cantidades y consultar artículos con bajo inventario.

Las pruebas permitieron identificar errores como:

- registros duplicados,
- cantidades negativas,
- fallas al actualizar archivos,
- inconsistencias al mostrar reportes.

Tras detectarlos, se ajustaron las funciones y se reforzaron validaciones para garantizar que la información del inventario permaneciera íntegra y consistente.

7. Documentación y mejora continua

Durante el desarrollo se elaboró documentación sobre el funcionamiento de cada módulo, instrucciones para el usuario y recomendaciones para mantenimiento. Esta documentación facilita que el sistema pueda ser ampliado en el futuro por el mismo equipo o por otros desarrolladores.

Asimismo, se elaboraron propuestas para futuras mejoras, como:

- integración con código de barras,
- generación de gráficos,
- módulo de usuarios con permisos,
- migración a base de datos SQL,
- implementación de un punto de venta completo.

Conclusiones

Tras la finalización del desarrollo y las pruebas necesarias del sistema de gestión de inventarios asistido por chatbot, se puede concluir que la hipótesis planteada al inicio de este proyecto ha sido cumplida. La validación funcional del software demostró que la interacción a través de un chatbot agiliza los procesos de consulta y modificación de la base de datos, permitiendo a los usuarios obtener información sobre existencias y ubicación de productos sin la necesidad de navegar por múltiples menús o ventanas complejas. Esta inmediatez en la respuesta válida la premisa de que la automatización de interfaces mediante el procesamiento de lenguaje natural es una herramienta viable y superior para entornos donde el “orden” es de suma importancia.

Uno de los aspectos más destacados del proyecto fue la exitosa implementación de la lógica de negocio del inventario traducida a comandos interpretables por el chatbot. Se logró construir una arquitectura robusta donde el backend gestiona eficazmente las transacciones, asegurando que no existan inconsistencias en el stock, independientemente de si la orden proviene de la interfaz gráfica estándar o del chat. Esto resalta la importancia de un diseño modular en el desarrollo de software, lo cual facilitó la integración de la API del chatbot sin comprometer la estabilidad del sistema central. Además, la experiencia de usuario se vio notablemente mejorada, las pruebas de uso indicaron que los sujetos de prueba percibieron el sistema como “intuitivo” y “moderno”, lo cual se podría considerar como un éxito en cuanto a la adopción de nuevas herramientas tecnológicas se trata.

Otro punto relevante a resaltar es la capacidad del sistema para manejar excepciones y errores de entrada. Al utilizar un chatbot, existía el riesgo de ambigüedad en las órdenes, sin embargo, los algoritmos de validación implementados permiten que el sistema solicite confirmación o aclaración antes de ejecutar cambios irreversibles en la base de datos, cumpliendo así con el objetivo de reducir los errores humanos. El proyecto no solo resultó en un producto funcional, sino que también sirvió para demostrar cómo es que las tecnologías de inteligencia artificial pueden transformar tareas administrativas monótonas en procesos dinámicos y eficientes.

Como trabajo a futuro, se sugiere la incorporación de algoritmos de aprendizaje automático (Machine Learning) para dotar al chatbot de capacidades predictivas. Esto permitiría al sistema no solo responder a comandos, sino sugerir reabastecimientos basándose en historiales de ventas y tendencias estacionales, convirtiéndose en una herramienta proactiva.

Finalmente, sería beneficioso escalar la accesibilidad del chatbot integrándolo con plataformas de mensajería como WhatsApp o Telegram lo que permitiría a los trabajadores gestionar el inventario con mayor facilidad y adaptabilidad, logrando un nuevo nivel de eficiencia.

Referencias bibliográficas

J. Heizer y B. Render, Principios de administración de operaciones, 9.^a ed. Naucalpan de Juárez, México: Pearson Educación, 2014. [En línea]. Disponible en: [Principles of Operations Management - Jay H. Heizer, Barry Render - Google Libros](#)

R. S. Pressman, Ingeniería del software: Un enfoque práctico, 7.^a ed. México D.F., México: McGraw-Hill, 2010. [En línea]. Disponible en: [Ingenieria del Software. Un Enfoque Practico](#)

E. Adamopoulou y L. Moussiades, "An Overview of Chatbot Technology," en Artificial Intelligence Applications and Innovations, vol. 584, 2020, pp. 373–383. [En línea]. Disponible en: https://doi.org/10.1007/978-3-030-49186-4_31

M. O. Ayomide et al., "A Review of Existing Inventory Management Systems," IJRES, vol. 12, no. 9, Sep. 2024.

W. C. Tan & M. S. Sidhu, "RFID IoT Architecture for Smart Inventory Management," ResearchGate, Feb. 2022.

"Application of Artificial Intelligence in Inventory Decision Op," PinnaclePubs, Aug. 20, 2025 (artículo sobre ML en inventarios).

"A Comprehensive Comparison of Two Leading ERP Systems," OpenSourceIntegrators, Jan. 30, 2024 (ERPNext vs Odoo).

"SAP Business One vs. QuickBooks: Which Solution is Right for ...," ConsensusIntl Blog, Jan. 15, 2025.